Reinforcement Learning

7

Pre

CS 3244
Machine Learning

NUS | Computing
National University of Singapore

Recap and Forecast

Understand how validation and regularization complement to address overfitting:

- **Regularization:** a means of restraining the model.
  - L^p norm forms of regularization.

$J_{test}(h) = J_{train}(h) +$ overfit penalty

- **Validation**, as a reality check by peeking (at the bottom line).
  - Cross Validation variants: k-fold or leave one out.

Learning Outcomes for this week:

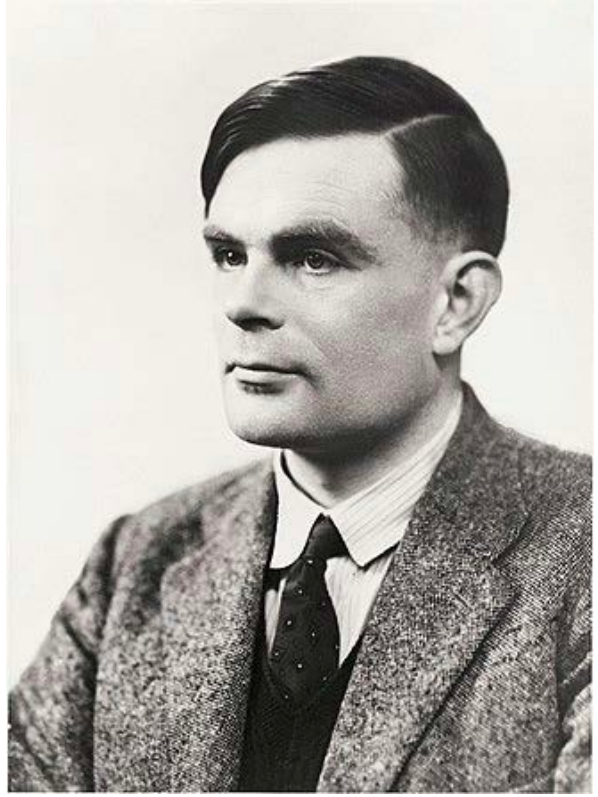
- Understand the basic terminology of RL: Policy, reward, action, observation and (environment, agent) model.
- Relate how a **Markov Decision Process** (MDP) embodies RL concepts.
- Understand the **Bellman equations** as a recursive form for calculating future rewards and state values.
- Solve planning and RL problems using the **Value Iteration** and **Q Learning** algorithms.
- Differentiate Value Iteration for planning from Q Learning for RL by their modeling assumptions

Characteristics of Reinforcement Learning

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

General Learning Algorithm



Source Image: Wikipedia

"Instead of trying to produce a program to simulate the adult mind, why not rather try to produce one which simulates the child's? If this were then subjected to an appropriate course of education one would obtain the adult brain."

– *Alan Turing*

Characteristics of Reinforcement Learning

What makes reinforcement learning different from other ML paradigms?

- There is no supervisor, only a reward signal
- Feedback is delayed, not instantaneous
- Time matters (sequential and non *i.i.d.* data)
- Agent's actions affect the subsequent data it receives

Screen capture from OpenAI.com YouTube video

Examples:

- Guide a helicopter to fly stunts
- Defeat humans at MMORPG
- Manage an investment portfolios
- Make robots walk or do stunts
- Control a power station

Rewards and the Reward Hypothesis

- A reward r_t is a scalar feedback signal
- Indicates how well the agent is doing by time step t
- The agent's job is to maximize cumulative reward

Reward Hypothesis

All goals can be described by the maximization of expected cumulative rewards

Quick Check: Assign rewards (+ve or –ve) to the scenarios on the previous slide

Sequential Decision Making

Goal: *select actions to maximise total <u>future</u> reward*

- Actions may have long term consequences
- Reward may be delayed

It may be better to sacrifice immediate reward to gain more long-term reward. Examples:

- A financial investment (may take months to mature)
- Participating in class (may help improve your own understanding just by asking)
- Blocking opponent moves (might help winning chances many moves from now)

The Agent and the Environment

Photo credits: Ryan Fields @ Unsplash

At each time step t, the agent:

- Receives scalar reward r_t
- Executes action a_t
- Receives observation o_t

The environment:

- Receives action a_t
- Emits observation o_{t+1}
- Emits scalar reward r_{t+1} for the next time step

t increments at each env. step

History and State

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

History and State

The history is the sequence of observations, actions, rewards

$$h_t = o_1, r_1, a_1, \dots, a_{t-1}, o_t, r_t$$

- *i.e.* all observable variables up to time t
- *i.e.* the sensorimotor stream of a robot or embodied agent

What happens next depends on the history:

- The agent selects actions
- The environment selects observations/rewards

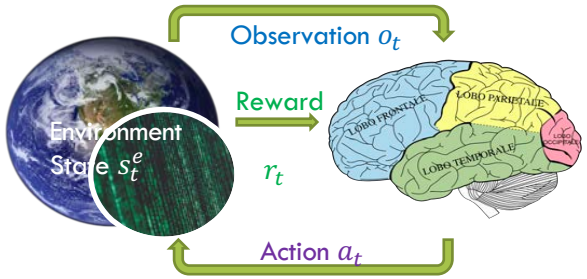
State is the information used to determine what happens next

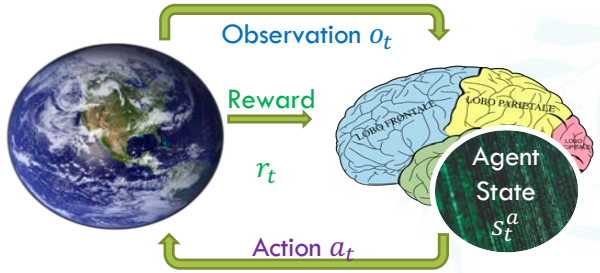
Formally, state is a function of the history:

$$s_t = f(h_t)$$

- Environment State
- Agent State
- Information State

Environment and Agent State





Environment state s_t^e = environment's private representation

- i.e. whatever data the environment uses to pick the next observation/reward
- The environment state is not usually visible to the agent
- Even if s_t^e is visible, it may contain irrelevant information

Agent state s_t^a = agent's internal representation

- i.e. whatever information the agent uses to pick the next action
- i.e. information used by the RL algorithm
- It can be any function of history:

$$s_t^a = f\,(h_t)$$

Logo credits: Vecteezy

Artwork from Wikipedia Commons: Earth / Brain; and Markus Spiske @ Unsplash

Markov Property

An information state (a.k.a. Markov state) contains all useful information from the history.

Definition: A state s_t is Markov if and only if

$$P(s_{t-1}|s_t) = P(s_{t+1}|s_1, \dots, s_t)$$

"The future is independent of the past given the present"

$$h_{1:t} \rightarrow \boldsymbol{s_t} \rightarrow h_{t+1:\infty}$$

Once the state is known, the history is irrelevant; *i.e.*,

- The state is a sufficient statistic of the future
- The environment state s_t^e is Markov
- The history h_t is Markov

А. А. Марков (1886).

Andrey Markov

Photo from Wikipedia Commons: Andrey Markov

Representing State: Training Rats

What if the agent state is represented by:

- last 3 items in sequence?
- counts for lights and bells?
- the complete sequence?

Photos all from Wikipedia Commons:

Fully Observable Environments

Screen Capture credit: Google DeepMind Challenge @ YouTube

Full observability: agent directly observes environment state:

$$o_t = s_t^a = s_t^e$$

Agent state = environment state = information state

This is a Markov decision process (MDP).

Partially Observable Environments

Agent indirectly observes environment:

- A vacuum robot with lidar isn't told its absolute location
- A trading agent only observes current prices
- A DOTA agent only observes their nearby context

Now, the agent state ≠ the environment state

Formally a partially observable Markov decision process (POMDP)

Screen capture from OpenAI.com YouTube video

Agent must construct its own state representation s_t^a, e.g.

- Complete history: $s_t^a = h_t$
- Beliefs of environment state: $s_t^a = (P[s_t^e = s_1], \ldots, P[s_t^e = s_n])$
- Recurrent neural network: $s_a = \sigma(s_{t-1}^a \Theta_s + o_t \Theta_o)$

Inside an RL Agent

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Primary Components of an RL Agent

Includes one or more of these components:

- Policy: the agent's behaviour function
- Value function: how good is each state and/or action
- Model: agent's representation of the environment

Policy π

A map from state s to action a, e.g.

- Deterministic policy: $a = \pi(s)$
- Stochastic policy: $\pi(a|s) = P(a_t = a|s_t = s)$

Value Function $v\ (s)$

A prediction of <u>future</u> reward

Used to evaluate the goodness/badness of states

Selects between actions, e.g.

γ : Discount for future rewards

$$v_\pi\ (s) = \mathbb{E}_\pi(r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots | s_t\ = s)$$

Model

A model predicts what the environment will do next

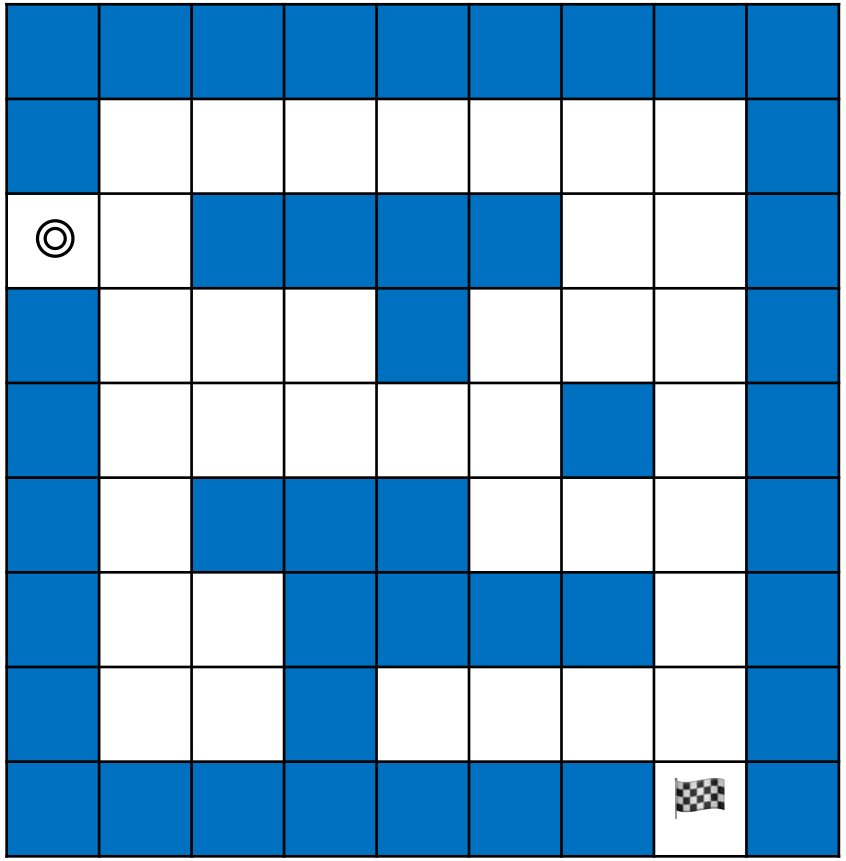
- $\mathcal{P}$ predicts the next state

$$\mathcal{P}_{s,s'}^{a} = P(s_{t+1} = s' | s_t = s, a_t = a)$$

- $\mathcal{R}$ predicts the next (immediate) reward

$$\mathcal{R}_{s}^{a} = \mathbb{E}(r_{t+1} | s_t = s, a_t = a)$$

GridWorld Maze

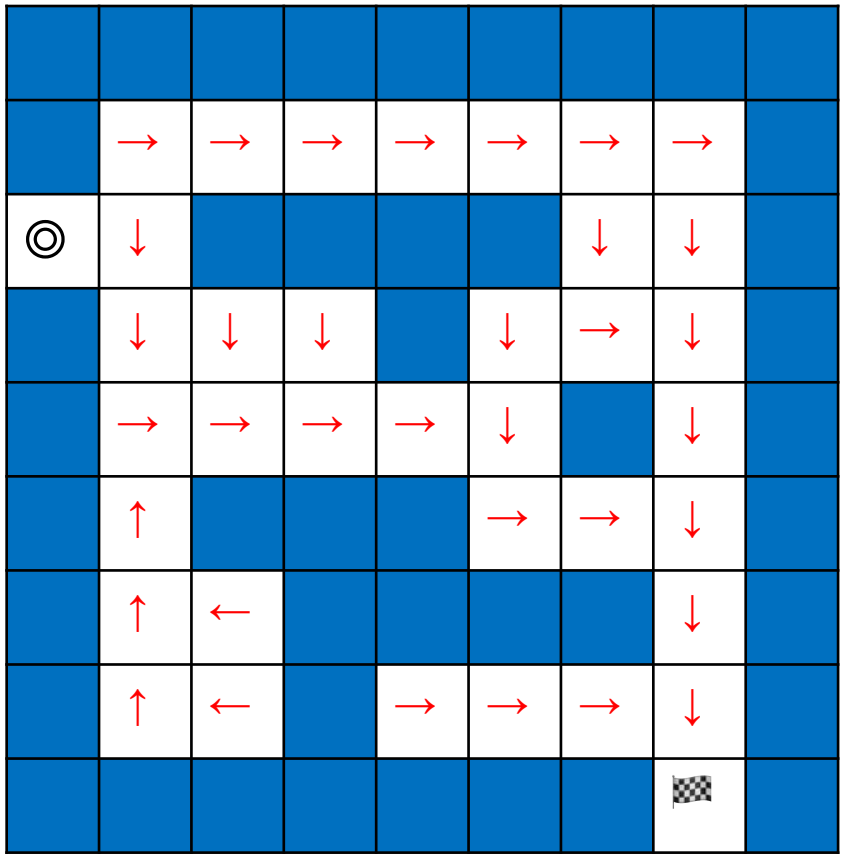


Rewards: –1 per time-step

Actions: `N, E, S, W`

States: Agent's location

GridWorld Maze: Policy



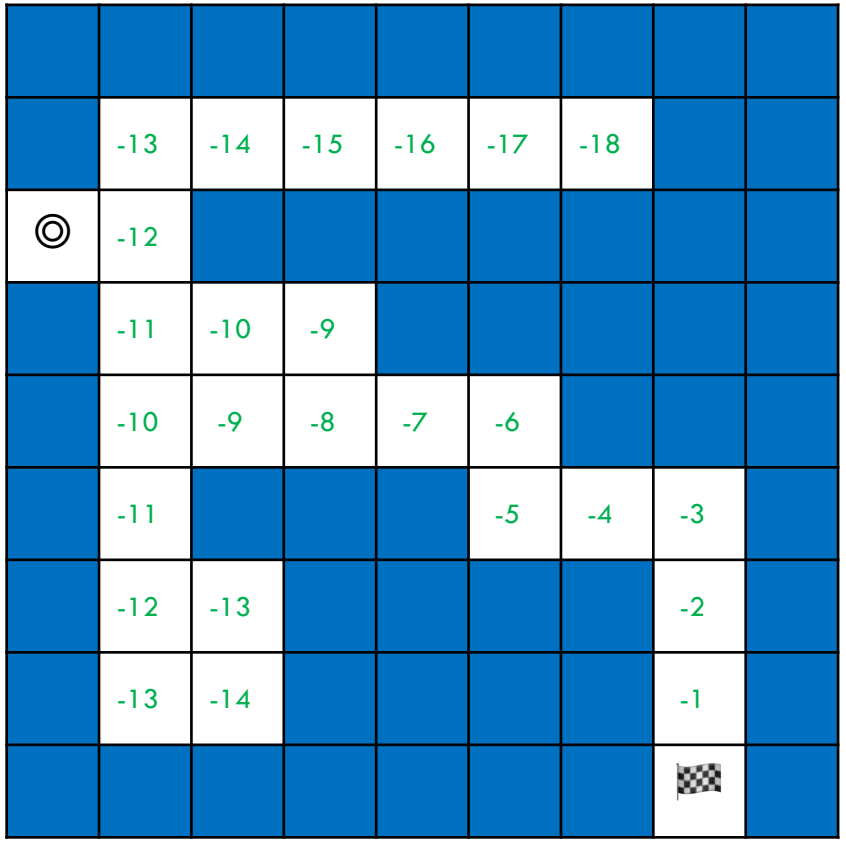
Arrows represent the policy $\pi(s)$ for each state s.

GridWorld Maze: Value Function

	-13	-12	-11	-10	-9	-8	-7	
◎	-12					-7	-6	
	-11	-10	-9		-7	-6	-5	
	-10	-9	-8	-7	-6		-4	
	-11				-5	-4	-3	
	-12	-13					-2	
	-13	-14		-4	-3	-2	-1	

Numbers represent the value function $v_\pi(s)$ for each state s.

GridWorld: Model

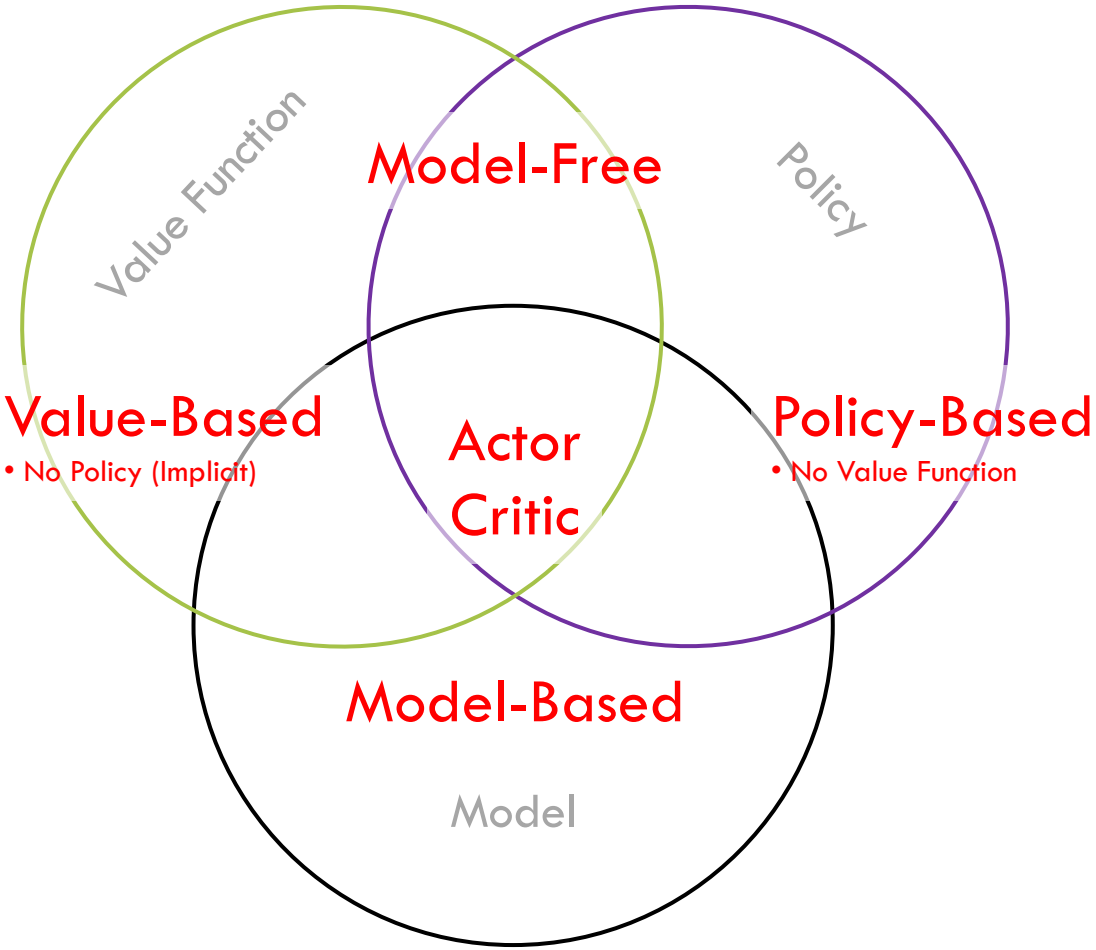


Agent may have an internal (imperfect) model of the environment

- Dynamics: how actions change the state
- Rewards: how much reward from each state

- Grid layout represents transition model $\mathcal{P}_{s,s'}^{a}$
- Numbers represent immediate reward $\mathcal{R}_{s}^{a}$ from each state s (same for all a)

RL Agent Taxonomy



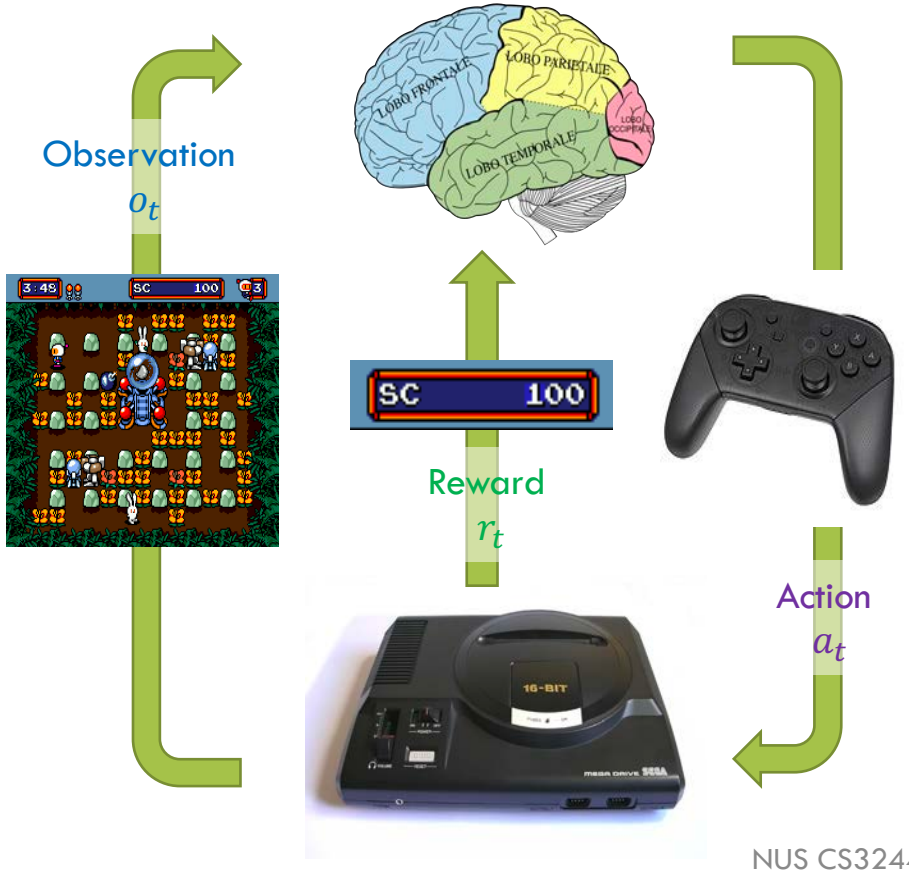
Learning and Planning

Two fundamental problems in sequential decision making:

- Reinforcement Learning:
  - The environment is initially unknown
  - The agent interacts with the environment
  - The agent improves its policy

- Planning:
  - A model of the environment is known
  - The agent performs computations with its **model** (without any external interaction)
  - The agent improves its policy
    a.k.a. deliberation, reasoning, introspection, pondering, thought, search

Reinforcement Learning Example



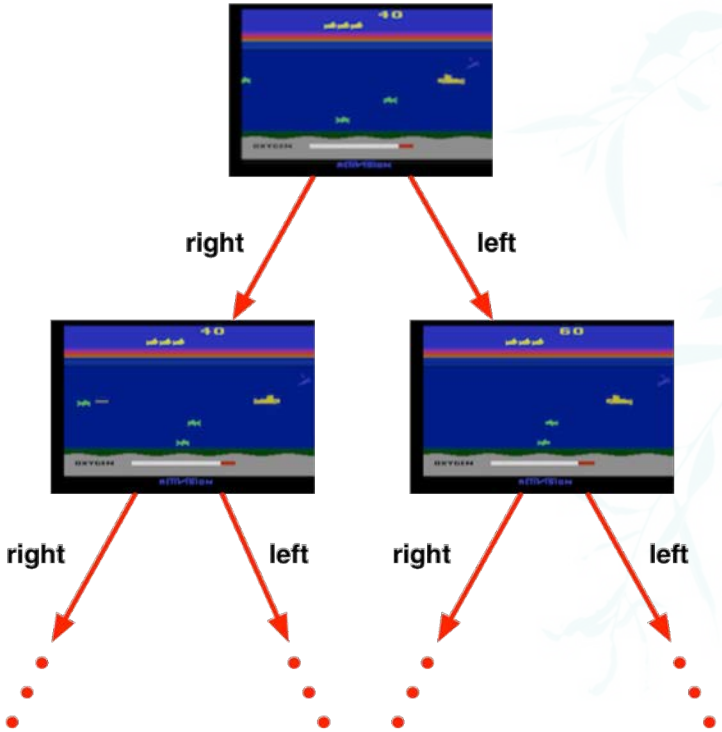
- Rules of the game are unknown
- Learn directly from interactive game-play
- Pick actions on controller, observe pixels and scores

Photos all from Wikipedia Commons:

Photos all from Wikipedia Commons:

Planning Example

- Rules of the game are known
- Can query emulator
  - perfect model inside agent's brain
- If I take action a from state s:
  - what would the next state be?
  - what would the score be?
- Plan ahead to find optimal policy
  - e.g. tree search, minimax



Markov Processes

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Preface 1: Markov Chains

A Markov process is a memoryless random process,
i.e., a sequence of random states $s_1, s_2, \ldots$ with the Markov property.

Definition:

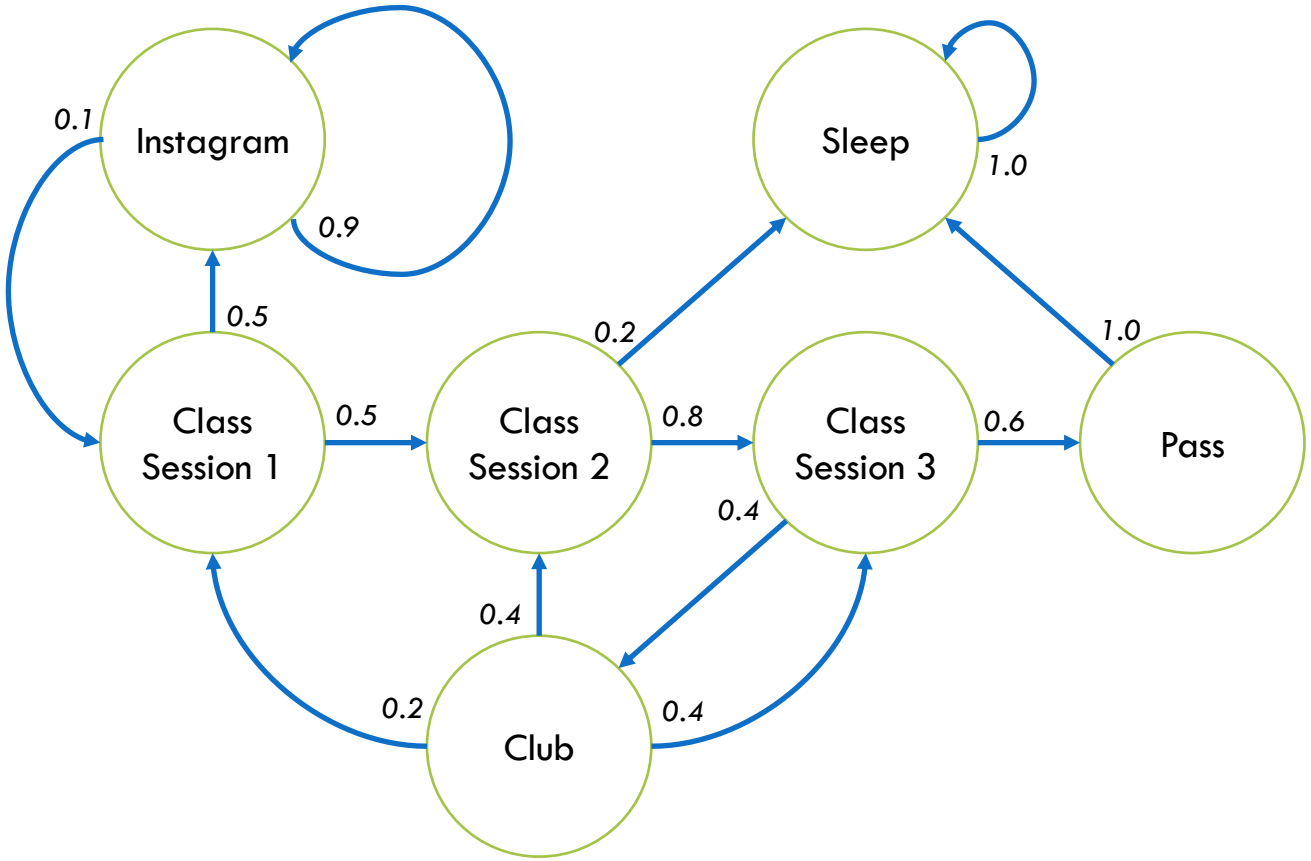
A *Markov Process* (or *Chain*) is a tuple $\langle \mathcal{S}, \mathcal{P} \rangle$

- $\mathcal{S}$ is a (finite) set of states
- $\mathcal{P}$ is a state transition probability matrix
  $\mathcal{P}_{s,s'} = P[\mathcal{S}_{t+1} = s' | \mathcal{S}_t = s]$

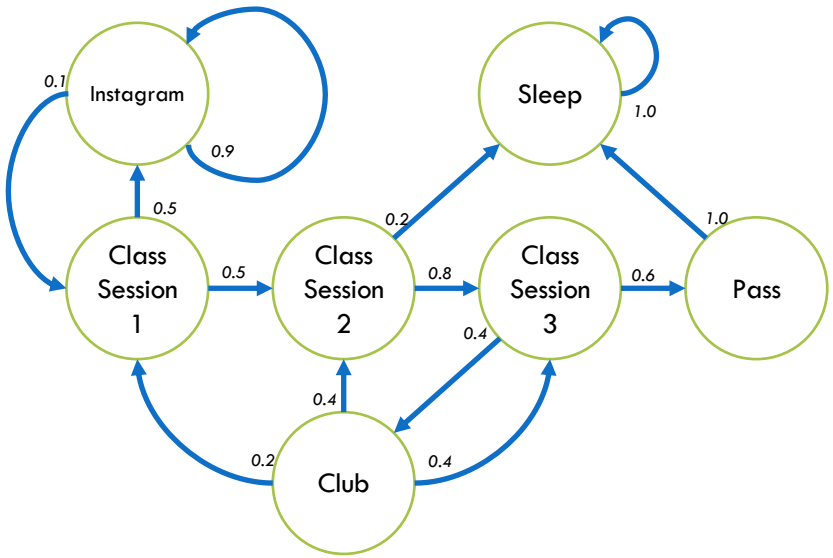
$$\mathcal{P} = \begin{bmatrix} \mathcal{P}_{1,1} & \cdots & \mathcal{P}_{1,n} \\ \vdots & \ddots & \vdots \\ \mathcal{P}_{n,1} & \cdots & \mathcal{P}_{n,n} \end{bmatrix}$$

Each row sums to 1

Student Markov Chain Example



Student Markov Chain Example



Sample episodes $s_1, s_2, \ldots s_T$ starting from s_1 = Class Session 1 (C1)

- C1 C2 C3 Pass Sleep
- C1 IG IG C1 C2 Sleep

From

To

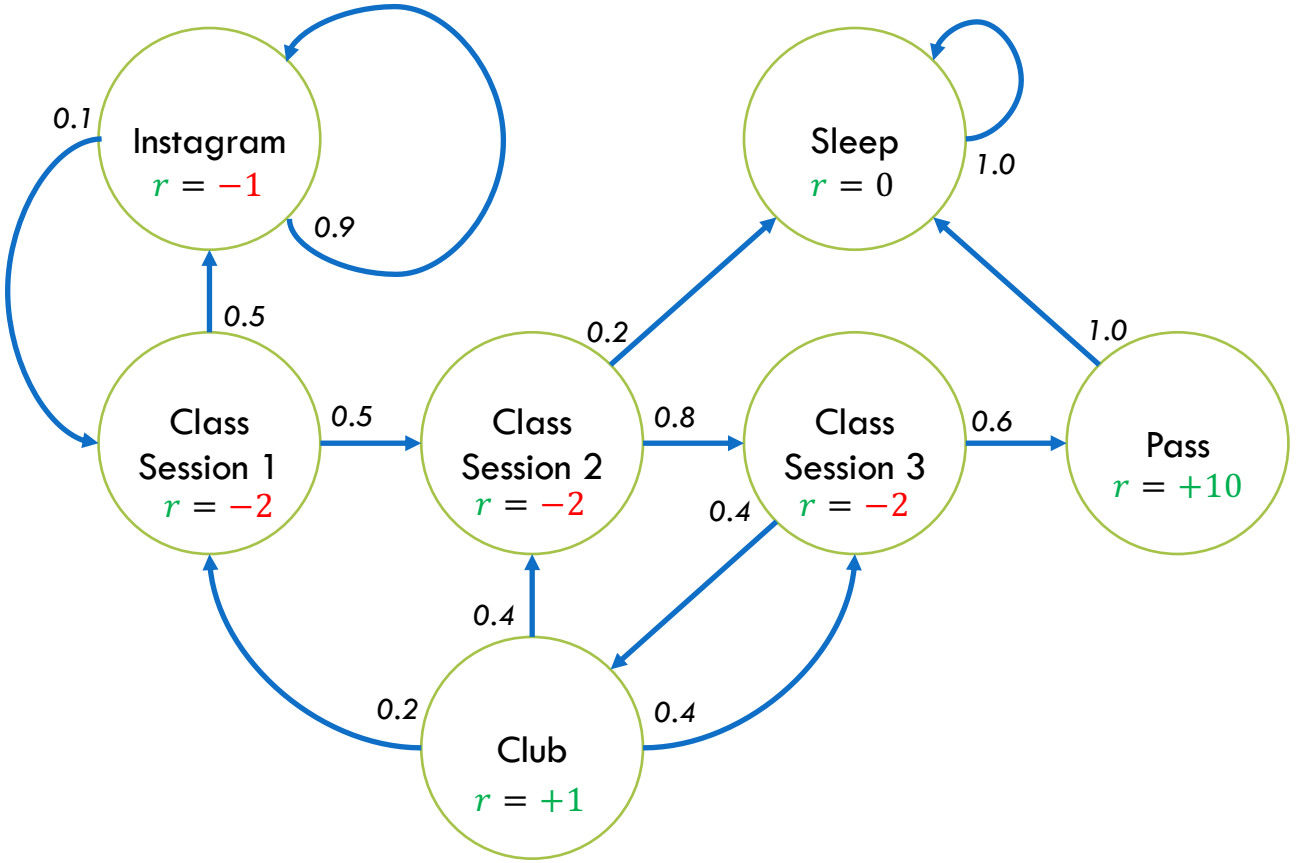
$$\mathcal{P} = \begin{array}{c} \\ C1 \\ C2 \\ C3 \\ Pass \\ Club \\ IG \\ Sleep \end{array} \begin{array}{c} \begin{array}{ccccccc} C1 & C2 & C3 & Pass & Club & IG & Sleep \end{array} \\ \begin{bmatrix} & 0.5 & & & & 0.5 & \\ & & 0.8 & & & & 0.2 \\ & & & 0.6 & 0.4 & & \\ & & & & & & 1.0 \\ 0.2 & 0.4 & 0.4 & & & & \\ 0.1 & & & & & 0.9 & \\ & & & & & & 1.0 \end{bmatrix} \end{array}$$

Preface 2. Markov Reward Process

A Markov chain with values. An MRP is a tuple $\langle \mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:

- $\mathcal{S}$ is a finite set of states
- $\mathcal{P}$ is a state transition probability matrix $\mathcal{P}_{s,s'} = P(s_{t+1=s'} | s_t = s)$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s = \mathbb{E}(r_{t+1} | s_t = s)$
- γ is a discount factor, $\gamma \in [0,1]$

Student Markov Reward Process



Markov Decision Process

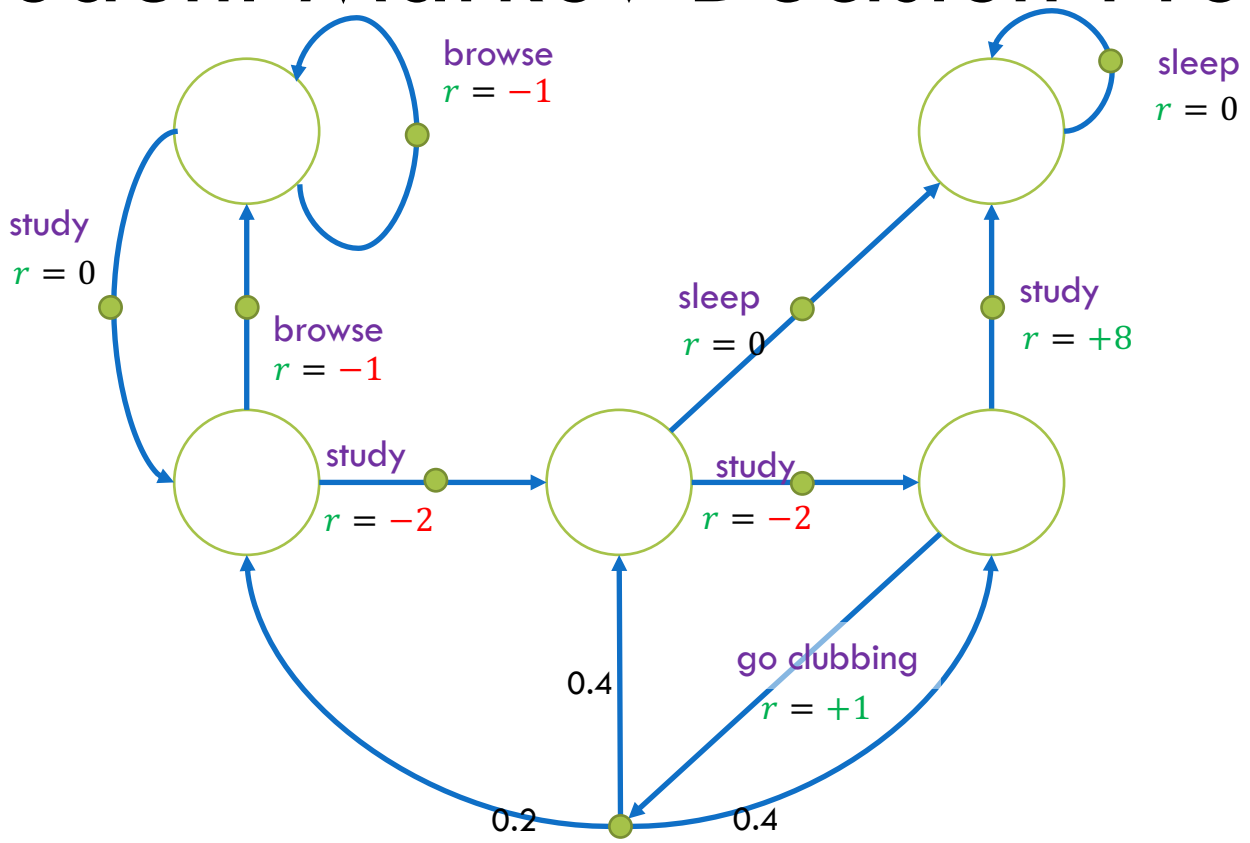
A Markov decision process (MDP) is a Markov reward process with decisions.

MDP $\equiv \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- $\mathcal{S}$ is a finite set of states
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P}$ is a state transition probability matrix, $\mathcal{P}_{s,s'}^{a} = P(s_{t+1} = s' | s_t = s, a_t = a)$
- $\mathcal{R}$ is a reward function, $\mathcal{R}_s^a = \mathbb{E}(r_{t+1} | s_t = s, a_t = a)$
- γ is a discount factor $\gamma \in [0, 1]$.

In MDP, rewards are indexed on action as well as state. Actions can lead to differing rewards.

Student Markov Decision Process



By convention, we leave out action nodes where there is only one outcome of the action.